TL 下排观测位 + 修剪新点位 · 新版说明文档 (V5)

适用：Nano 板上的独立验证包 `~/Desktop/TL_lowerobs_test`（修剪点位同时已同步到 `~/Desktop/TL`）

原则：教 / 跑用本验证包的脚本；旧流程不用时随时可回退

更新日期：2026-10-08（V5）

目 录

0. 版本更新记录 4

1. 新增功能总览 5

2. 命令总表 6

3. 每次启动前 7

4. 判定规则（盆号 → 排 → 侧 → 去哪里识别） 8

5. 详解①：下排观测位 与 示教端 `v / d / g` 9

    5.1 为什么需要 9

    5.2 为什么要分 A / B 两侧 9

    5.3 三个按键（种子示教、叉子示教行为一致） 9

    5.4 正确操作姿势 9

    5.5 数据存在哪 10

6. 详解②：运行端下排路由 11

    6.1 种子运行（mock_pick_place） 11

    6.2 叉子运行（fork_trim_mqtt / fork_basket_cycle） 11

    6.3 日志对照（怎么确认走没走观测位） 11

7. 详解③：两个固定修剪点位（TRIM_HOLD_1 / TRIM_HOLD_2） 12

7.1 用在哪（搬运链条） 12
7.2 为什么一个点位要写两份数据（共 4 行常量） 12
7.3 当前数值（2026-10-07 更新，示教器读数） 12
7.4 旧值（备用，如需回退） 13
7.5 以后怎么再改（Nano 一键替换） 13
7.6 文件位置与副本 14
8. 详解④：作业状态文件与"a basket is already held" 15
8.1 是什么 15
8.2 什么时候写、什么时候删 15
8.3 phase（阶段）含义 15
8.4 启动时的保护行为（正常设计，不是故障） 15
8.5 怎么解除（**先肉眼看叉上有没有花篮**） 16
8.6 纪律：动作没走完不要按 Ctrl+C 16
9. 新版【种子示教】操作步骤 17
第 0 步：记录两侧"下排观测位"（一次性，跑下排盆之前必须做） 17
第 1 步：教盆（每盆） 17
键位速查（种子示教） 17
10. 新版【叉子示教】操作步骤 19
第 0 步：一次性设置 19
第 1 步：教第一盆（含全局 W / T，必须教一次） 19
第 2 步：后续每盆 19
关于 W / T 和"固定值" 19
键位速查（叉子示教） 20
11. 运行端验证（闭环） 21
11.1 种子（放种） 21
11.2 叉子（叉取） 21
11.3 推荐的首次验证顺序 22
12. 常见提示 / 报错对照 23
13. 数据文件与回退 25
14. 操作纪律与快捷索引 26
15. 修正：A3 排码贴反（`--reversed-rows A3`） 27

（首次打开：右键目录 → 更新域 → 更新整个目录）

0. 版本更新记录

版本	日期	更新内容
V1	2026-10-06	新增"下排观测位"（A/B 分侧）：示教端 `v / d / g` 三个键；运行端自动路由
V2	2026-10-07	① **修剪位 1 / 2 两个固定点位**换成新示教器读数（第 7 章）② 新增**作业状态文件**说明与"花篮未放回"的恢复方法（第 8 章）③ 补充运行流程阶段详解、Ctrl+C 纪律、新报错对照（第 11 / 12 章）
V3	2026-10-08	下排盆（A1/A4/B1/B4）的 **W/T 改为独立槽位**（第 5 章）：示教下排盆时按 W/T 自动存"下排专用"；运行下排盆自动用下排那对（pick 与返回路径都按盆的排选择；未教前沿用上排并打印 WARNING）
V4	2026-10-08	**A3 排 ArUco 码贴反修正**：4 个脚本内置 `--reversed-rows A3`（输入的位置号自动镜像为该格实际贴的码：A3-01↔ID71 … A3-24↔ID48）。已教 A3 盆的命令变化、补教方法与日志对照见新增第 15 章
V5	2026-10-08	① **`O / v / g` 三个"位"分工写清楚**（第 10 节键位速查后）：`O`=待机位（= 种球默认位，不是看花篮的位置）、`v`=A/B 起始位（-83.668… / 96.332…，老的观测位）、`g`=下排观测位 ② **ID ↔ 编号换算**补进第 4 节（ID34 → 编号 35 → A2-11）

1. 新增功能总览

#	新功能	一句话说明	详解
1	下排观测位（A/B 分侧）	下排盆（A1/A4/B1/B4）改用专用"观测位"识别码，分 A、B 两侧各记一个	第 5 章
2	示教端 `v / d / g` 按键	切侧 / 现场记录 / 回位验证（种子、叉子两套示教都有）	第 5 章
3	运行端下排自动路由	下排盆先到对应侧观测位识别码；上排盆行为与旧版完全一致	第 6 章
4	修剪固定点位（位置 1 / 2）	两个"写死在代码里"的点位换成新读数	第 7 章
5	作业状态文件	防"花篮没放回就再抓"的安全记账（`active_transfer.yaml`）	第 8 章

改动的文件（均在验证包 `src/` 内）：

文件	改动
`tl_hand_deploy/aruco_basket_teach.py`	种子示教：新增 `v` / `d` / `g`
`tl_hand/mock_pick_place.py`	种子运行：下排盆先去观测位识别（调用手爪的部分未动）
`tl_fork_teach/fork_basket_teach.py`	叉子示教：新增 `v` / `d` / `g`（`d` 由旧版"去固定参数位姿"改为"现场拖到位记录"）
`tl_fork_teach/fork_basket_cycle.py`	叉子运行：下排路由 + 修剪位 1 / 2 新数值（trim 节点自动继承）

2. 命令总表

用途	旧命令（保留不动）	新版命令（本次用这个）
种子示教	`bash ~/Desktop/TL/scripts/teach_seed.sh`	`bash ~/Desktop/TL_lowerobs_test/scripts/teach_seed.sh`
叉子示教	`bash ~/Desktop/TL/scripts/teach_fork.sh`	`bash ~/Desktop/TL_lowerobs_test/scripts/teach_fork.sh`
种子运行（放种）	`bash ~/Desktop/TL/scripts/run_seed_mqtt.sh`	`bash ~/Desktop/TL_lowerobs_test/scripts/run_seed_mqtt.sh`
叉子运行（叉取）	`bash ~/Desktop/TL/scripts/run_fork_mqtt.sh`	`bash ~/Desktop/TL_lowerobs_test/scripts/run_fork_mqtt.sh`

版本选择建议（两版共用同一份示教数据，绝不能同时运行）：**上排盆的种子抓放推荐用老版 `~/Desktop/TL`**（成熟稳定）；**下排盆（A1/A4/B1/B4）与叉子流程用新版** —— 老版没有"下排观测位"功能。

3. 每次启动前

```
# ① 驱动（另一个终端，照旧）
ros2 launch tl_driver tl_tcb605_driver.launch.py

# ② 脚本权限（从 Windows 拷过去会丢执行位，执行一次即可；不想改权限就一律用 bash 前缀跑）
chmod +x ~/Desktop/TL_lowerobs_test/scripts/*.sh

# ③（可选）确认修剪点位是新数值（应显示 372.383 / 371.473 等）
grep -n "TRIM_HOLD"
~/Desktop/TL_lowerobs_test/src/tl_fork_teach/tl_fork_teach/fork_basket_cycle.py
```

怎么确认跑的是新版（启动打印里出现下面字样即正确）：

- 种子示教：`only v / g / R keys send robot motion` ＋ `Current side: A …`
- 叉子示教：`New keys: v switch side | d record lower-observation pose | g goto it.` ＋ `Current side: A …`

安全纪律：E-stop 放在手边；同一时间只开一个会发运动的程序；**新旧程序不要同时跑**（MQTT client-id、ServoJ 话题会冲突）；**动作没走完不要按 Ctrl+C**（见第 8.6 节）。

4. 判定规则（盆号 → 排 → 侧 → 去哪里识别）

排号 =（盆号 − 1）÷ 24 → 0..7 对应 A1 A2 A3 A4 B1 B2 B3 B4。

（ArUco 码 ID 与盆号换算：**盆号 = ID + 1**；例 ID34 → 盆号 35 → A2-11。）

盆号	排	侧	运行时识别位置
1–24	A1	A	**下排观测位 A**
25–48	A2	A	原方式（该盆检索位）
49–72	A3	A	原方式
73–96	A4	A	**下排观测位 A**
97–120	B1	B	**下排观测位 B**
121–144	B2	B	原方式
145–168	B3	B	原方式
169–192	B4	B	**下排观测位 B**

上排 = A2 / A3 / B2 / B3；下排 = A1 / A4 / B1 / B4。

5. 详解①：下排观测位 与 示教端 `v / d / g`

5.1 为什么需要

下排盆（A1 / A4 / B1 / B4）位置低，站在原来的"检索位"相机看不清盆上的码。所以给下排盆增加一个专用"观测位"：机械臂**先去观测位识别码**，识别成功后再去该盆的检索位执行动作（放种 / 叉取）。上排盆不需要，一切照旧。

5.2 为什么要分 A / B 两侧

A、B 两区机械臂是"转 180°"的关系（B 侧 = A 侧 J1 + 180°），一个观测位无法同时看清两侧下排，所以每侧各记录一个自己的观测位。

5.3 三个按键（种子示教、叉子示教行为一致）

键	动作	会动机械臂吗	终端打印（示例）
v	切到另一侧：臂自动去该侧起始位（A 起始位 ⇄ B 起始位），"当前侧"随之改变	✅ 会	`Current side: B (d records / g goes to the B lower observation pose)`
d	把**按 d 这一刻的臂位姿**记录为当前侧的下排观测位（重复按 = 覆盖）	❌ 不动	`Saved lower observation pose for side A: joint=[…]`
g	臂自动回到当前侧已记录的观测位（验证记录、也方便接着拖）	✅ 会	`Moving to side A lower observation pose …`

> 两个示教程序的启动横幅都提示：叉子示教"只有 v / g / R 会自己动"；种子示教"只有 v / g / R 会自己动"。其余全靠手拖。

5.4 正确操作姿势

```
拖到位 → 松手停稳 1~2 秒 → 按 d
   （若提示 /tcp_pose is stale：说明臂没停稳，再等 1~2 秒重按 d）
→ 按 g 自动回位验证（臂能回到刚记录的位置就说明存对了）
→ 记错侧：按 v 回正确侧 → 重新拖到位 → 再按 d 覆盖
```

每次按完 d，核对打印里的 side=A/B 与你的预期是否一致。

5.5 数据存在哪

程序	文件	键名
种子	`~/tl_mock_pick_place/basket_aruco.yaml`	`lower_observation_pose_a` / `lower_observation_pose_b`
叉子	`~/tl_fork_teach/fork_basket_teach.yaml`	`lower_observation_pose_a` / `lower_observation_pose_b`

每条记录包含：`side`（A/B）、`joint`（6 个关节角）、`tcp`、记录时间。**旧程序不读这两个键**，所以对新功能不满意时旧流程照用不受影响。

6. 详解②：运行端下排路由

6.1 种子运行（mock_pick_place）

收到 `seed` 命令 → 解析 targetId 得到盆号 → 算出排号：

- **下排盆（A1/A4/B1/B4）**：先去对应侧的观测位 → 在那里识别码 → 再去该盆检索位执行放种；
- **上排盆**：照旧直接去该盆检索位识别（与旧版一模一样）。

取值优先级：启动参数 `--lower-observation-joints-a/-b`（一般不用）> YAML 里示教记录的观测位 > 都没有则报错（retCode 3：`lower observation pose for side X is not recorded`）。

6.2 叉子运行（fork_trim_mqtt / fork_basket_cycle）

`trim_pick` 时同样先判定：

- 下排盆 → 先去对应侧观测位识别，再去检索位叉取；
- 上排盆 → 检索位识别（同旧版）。

⚠ 注意区分两个"观测位"：

- **下排观测位**（本章说的）：`v/d/g` 现场记录的、只给下排盆识别码用；
- **待机观测位**（`O` 键记录 `observation_pose`）：每次作业结束后回位的站立位，上/下排都用。

6.3 日志对照（怎么确认走没走观测位）

场景	日志片段
种子·下排	`Basket number=N is on a lower row (A1/A4/B1/B4); moving to the side A lower observation pose to detect the code.`
种子·上排	`Moving to the taught search pose for basket number=N.`
叉子·下排	`Basket number=N: localizing marker at the lower-row observation pose...`
叉子·上排	`Basket number=N: localizing marker at search pose...`

7. 详解③：两个固定修剪点位（TRIM_HOLD_1 / TRIM_HOLD_2）

7.1 用在哪（搬运链条）

```
trim_pick:  抬升 L → W（示教）→ T（示教）→ 位置 1（高位过渡）→ 位置 2（修剪低位，等剪根）
trim_place: 位置 2 →（垂直提起回翻）→ 位置 1 → T → W → L → I → F → 检索位 → 待机观测位
```

- **位置 1**：到达 T 之后的高位过渡点（绕 J1 回转，叉保持水平，花篮安全）——从 T **直接关节驱动**过去；
- **位置 2**：修剪作业低位（花篮放低供车端剪根）——从位置 1 **先在高处平移到位置 2 的正上方（保持位置 1 的水平姿态）→ 再低速垂直下降 + J5 大幅翻转到位**；返回时对称走。

7.2 为什么一个点位要写两份数据（共 4 行常量）

常量	用途
`TRIM_HOLD_x_JOINT_DEG`（6 个关节角）	T→位置 1 的关节直达；位置 1→位置 2 直线走不通时的兜底插值
`TRIM_HOLD_x_TCP`（X,Y,Z + A,B,C）	位置 1→位置 2 的"高空平移 + 垂直下降"按**直角坐标**规划，需要位置和姿态

所以"改了 2 个点"在代码里就是 **4 行**（2 点 × 关节/直角各 1 行），必须成对替换，否则直线段还在用旧坐标。

7.3 当前数值（2026-10-07 更新，示教器读数）

常量	数值
`TRIM_HOLD_1_JOINT_DEG`	`-1.402, 2.199, -1.109, -18.450, -87.567, 21.143`
`TRIM_HOLD_1_TCP`	`372.383, -11.174, 512.929, 2.081, 1.527, 1.107`
`TRIM_HOLD_2_JOINT_DEG`	`-10.992, -51.448, -35.678, -12.270, -175.183, 6.713`
`TRIM_HOLD_2_TCP`	`371.473, -39.149, 175.217, -2.587, 1.529, -0.448`

取值口径：**关节取示教器"关节坐标 J 表"；直角取"大地坐标 WX/WY/WZ/WA/WB/WC 表"**（与代码原注释一致；其余几张表差别 ≤0.01mm，可忽略）。

7.4 旧值（备用，如需回退）

常量	旧值
TRIM_HOLD_1_JOINT_DEG	-3.164, 5.379, -1.368, -2.153, -92.470, -1.274
TRIM_HOLD_1_TCP	486.657, -48.233, 415.655, 0.440, 1.445, 2.637
TRIM_HOLD_2_JOINT_DEG	-3.051, -57.827, -34.822, 1.369, -178.869, 4.907
TRIM_HOLD_2_TCP	495.720, -50.561, -8.761, 2.279, 1.469, 0.949

7.5 以后怎么再改（Nano 一键替换）

在 Nano 终端整段粘贴运行（会自动找到所有 `TL*` 目录里含旧值的 `fork_basket_cycle.py` 并替换、核对）：

```
python3 - <<'PYEOF'
from pathlib import Path
import os

pairs = [
    # 旧行 → 新行（要再改时，把这两列换成你的数值）
    ("TRIM_HOLD_1_JOINT_DEG = [-1.402, 2.199, -1.109, -18.450, -87.567, 21.143]",
     "TRIM_HOLD_1_JOINT_DEG = [-1.402, 2.199, -1.109, -18.450, -87.567, 21.143]"),
    ("TRIM_HOLD_2_JOINT_DEG = [-10.992, -51.448, -35.678, -12.270, -175.183, 6.713]",
     "TRIM_HOLD_2_JOINT_DEG = [-10.992, -51.448, -35.678, -12.270, -175.183, 6.713]"),
    ("TRIM_HOLD_1_TCP = [372.383, -11.174, 512.929, 2.081, 1.527, 1.107]",
     "TRIM_HOLD_1_TCP = [372.383, -11.174, 512.929, 2.081, 1.527, 1.107]"),
    ("TRIM_HOLD_2_TCP = [371.473, -39.149, 175.217, -2.587, 1.529, -0.448]",
     "TRIM_HOLD_2_TCP = [371.473, -39.149, 175.217, -2.587, 1.529, -0.448]"),
]

desktop = Path.home() / "Desktop"
targets = []
for base in sorted(desktop.glob("TL*")):
    if not base.is_dir():
        continue
    for root, dirs, files in os.walk(base):
        dirs[:] = [d for d in dirs if d != "__pycache__"]
        if "fork_basket_cycle.py" in files:
            targets.append(Path(root) / "fork_basket_cycle.py")

for f in targets:
    text = f.read_text(encoding="utf-8")
    hit = 0
    for old, new in pairs:
        if old in text:
            text = text.replace(old, new)
            hit += 1
    if hit:
        f.write_text(text, encoding="utf-8")
        print(f"已更新 {hit}/4 处 -> {f}")
    else:
```

```
        print(f"未发现旧数值（可能已更新过）-> {f}")

print()
print("===== 核对：各副本当前的 TRIM_HOLD 数值 =====")
for f in targets:
    print(f)
    for line in f.read_text(encoding="utf-8").splitlines():
        if line.startswith("TRIM_HOLD"):
            print("   ", line)
PYEOF
```

上面脚本的"旧行"目前填的是**当前值**（用于核对、无实际修改）。将来要换新点位时：把"旧行"改成现在的值、"新行"改成新值，再整段运行即可。

注意：

- 改完**不需要重新编译**（运行脚本是 `PYTHONPATH` 直读 `src` 源码），**重启运行程序后生效**；示教程序不受影响，不用重新示教其他数据。
- 首次测试盯着 `carry: T->P1` → `carry: P1->P2 high translate` → `carry: P2 lower: vertical`；若出现 `WARNING: … falling back to the joint interpolation path` 是自动兜底，偶发正常。
- 新点位与旧值差异较大（位置 1：X 486→372、Z 416→513；位置 2：X 496→371、Z −9→175，修剪低位高了约 184mm）——换点后先空载确认高度符合车端剪根需求。

7.6 文件位置与副本

常量在 `src/tl_fork_teach/tl_fork_teach/fork_basket_cycle.py` **顶部第 32–35 行**。已同步更新的副本：

- `~/Desktop/TL_lowerobs_test/src/...`（验证包，本次测试用）
- `~/Desktop/TL/TL_Handback/src/...`（主程序）

（桌面另有 8 月旧快照 `~/Desktop/TL_Handback` 及其 `install/build` 编译产物，运行脚本不会读它们，无需处理。）

8. 详解④：作业状态文件与"a basket is already held"

8.1 是什么

文件：`~/tl_fork_teach/active_transfer.yaml`。运行程序把"**正在搬运的花篮**"记在里面（程序文档里叫 active fork transfer state）：

```
active: true
marker_id: 29          # = 30 号盆
phase: hold1           # 当前阶段
# …以及这台盆的取放路径关节（approach/insert/lift/search 等）
```

8.2 什么时候写、什么时候删

时机	动作
`trim_pick` 抬升成功后	开始写入；每到一个阶段刷新（`lifted` → `temporary` → `hold1` → `hold2`）
`trim_place` 走完返回、花篮放回后	**自动删除**（= 清空，"没有进行中的作业"）
任意阶段中途被 Ctrl+C / 强制停止	⚠ **残留**——代表"花篮可能还在叉上没放回"

8.3 phase（阶段）含义

phase	含义
`lifted` / `temporary`	已抬升 / 已到暂放位 T
`hold1`	已到修剪位置 1
`hold2`	已到修剪位置 2（等你发 `trim_place`）
`waypoint` / `inserted` / `retracted`	返回途中：回到 W / 插入位 / 退出位

8.4 启动时的保护行为（正常设计，不是故障）

发现残留状态时，运行程序会打印：

```
Active fork transfer state found: basket number=30, phase=hold1. Press R only if the arm
is physically at the saved phase.
Startup: a basket is still held (active fork state); the arm will NOT auto-move. Send
trim_place to return the basket first.
```

- **不自动回待机位**（避免沿"没示教过的路径"拖动可能还挂着的花篮）；
- **拒绝新的 trim_pick**：`retCode 2`，`a basket is already held; send trim_place first`。

8.5 怎么解除（**先肉眼看叉上有没有花篮**）

现场情况	处理
叉是空的（多数情况）	直接清掉这个"记账文件"： `cat ~/tl_fork_teach/active_transfer.yaml`（看一眼确认） `rm ~/tl_fork_teach/active_transfer.yaml` 它只是运行程序的"进行中作业记账本"，**示教数据（fork_basket_teach.yaml）不受影响**。之后重启运行程序，启动时不再出现 8.4 的两条提示。
叉上真挂着花篮	**不要删**！重启运行程序后发 MQTT： `{"cmd":"trim_place","targetId":"<对应盆>"}` 让它沿示教路径走完返回、把花篮放回（完成会自动清状态并回待机位）。

8.6 纪律：动作没走完不要按 Ctrl+C

为什么会有残留状态？绝大多数是**搬运 / 回位动作还在走的时候按了 Ctrl+C**：程序被关闭、动作停在半路、状态文件没来得及清。

- 想停程序：等它**空闲**（没有 `carry:` / `return:` / `Returning with …` 之类的动作输出）再停；
- 想看"走完了没有"：等 `Returned: max joint error=… deg` / `Cycle complete …` 这类完成打印；
- 真急停：用**物理 E-stop**（或 MQTT `estop` 命令），不要用 Ctrl+C 当急停。

9. 新版【种子示教】操作步骤

```
bash ~/Desktop/TL_lowerobs_test/scripts/teach_seed.sh
```

启动后自动运动到 A 侧起始位；窗口显示 `visible=… selected=… side=A`。

第 0 步：记录两侧"下排观测位"（一次性，跑下排盆之前必须做）

```
A 侧：
 1. 按住物理拖拽钮，把臂拖到"能清晰、稳定看到 A 区下排码（A1/A4 排）"的位置
 2. 松手，等 1~2 秒（臂停稳）→ 按 d
    → 打印：Saved lower observation pose for side A: joint=[…]        ✅
    → 若提示 "/tcp_pose is stale"：再等 1~2 秒按一次 d
 3. 按 g 验证：臂自动回到刚记录的位置（到位后自动释放 ServoJ，可继续拖拽）
B 侧：
 4. 按 v → 臂自动切到 B 侧起始位，打印 Current side: B
 5. 拖到能看清 B 区下排码（B1/B4 排）的位置 → 停稳 → 按 d → Saved … side B
 6. 按 g 验证回位
```

记错侧：按 `v` 切到正确侧、重新拖到位、再按 `d` 覆盖。**每次按完 d 看一眼打印的 side=A/B 对不对。**

第 1 步：教盆（每盆）

```
① 拖到能看清目标盆码的位置（下排盆可以先按 g 到该侧观测位附近再找码）
② 输编号 1..192（回显 Basket number buffer: …；c 清空）→ 按 B
   → 打印 Selected basket number=X …
   ⚠ 臂必须停稳（等 1~2 秒）；报 "not visible / not enough stable samples" 就停稳后重按
③ 拖到落种接近位 → 按 A
④ 拖到落种释放位 → 按 P（会打印落差 mm）
⑤ 按 R 回检索位 → 拖到下一盆，重复 ①~⑤
```

键位速查（种子示教）

键	作用	新旧
数字 / c	编号缓冲 / 清空	同
B	选中（冻结码 + 存检索位）	同
A / P	落种接近位 / 释放位	同
F / I / L / T	叉子位姿（备用）	同
d	**记录当前位姿为"当前侧"下排观测位**	🔄 变化（原来是"选中"别名）
g	**回当前侧下排观测位（验证用）**	🆕 新增
v	**A/B 侧来回切换（去该侧起始位）**	🔄 增强（原来只能 A→B）
R	回检索位 + 释放 ServoJ	同
Q	退出	同

R 是一次自动运动（从当前位置回"当前选中盆"的检索位，几秒~十几秒）：**等打印 `Returned: max joint error=… deg` 才算走完**；期间别按键、别 Ctrl+C。

10. 新版【叉子示教】操作步骤

```
bash ~/Desktop/TL_lowerobs_test/scripts/teach_fork.sh
```

启动**不动臂**；只有 `v / g / R` 会自己动，其余全靠手拖；窗口 HUD 带 `side=A`。

第 0 步：一次性设置

```
1. 按 O → 臂去**待机位**（2026-10-07 起统一为种球默认位 `[-23.246, …]`）并记录（运行端启动/干完活回来的位置；建议做一次，把旧值刷新）
2. A 侧：拖到能看清 A 区下排码处 → 停稳 → 按 d → Saved … side A → 按 g 验证
3. （可选）按 v → 切到 B 侧起始位（96.332…）→ 拖到位 → 按 d → 按 g 验证（**不记录也可以**：运行端会自动按 A 侧 +180° 推导，2026-10-07 起）
```

第 1 步：教第一盆（含全局 W / T，必须教一次）

```
① 输编号 1..192 → 按 B（冻结码 + 存检索位；臂停稳，否则等 1~2 秒重按）
② 拖到叉接近位          → 按 F
③ 水平插入到盆沿下      → 按 I（校验：水平 ≥20mm、Z 漂 ≤60mm）
④ 竖直抬起              → 按 L（校验：竖直 ≥30mm、XY 漂 ≤60mm）
   （F→I、I→L 姿态变化 ≤25°）
⑤ 拖到高位载运点        → 按 W（相对 L 姿态 ≤25°）
⑥ 拖到暂放位（修剪位附近）→ 按 T（L→T 水平 ≥30mm、W→T 姿态 ≤120°）
⑦ 按 R 回检索位（ServoJ 释放）→ 下一盆
```

第 2 步：后续每盆

```
输编号 → B → F → I → L → R          （W / T 是全局位姿，不用再按；重按会覆盖，别误按）
```

关于 W / T 和"固定值"

位姿	来源	频率	
L 抬升位	示教（L 键）	**每盆**	
W 载运点	上排 `carry_waypoint`；下排专用 `carry_waypoint_lower`	示教（W 键）	**每类一次**：上排一对；下排盆按 W 会自动存"下排专用"（2026-10-08 起）
T 暂放位	上排 `fork_transfer_pose`；下排专用 `fork_transfer_pose_lower`	示教（T 键）	同上
修剪位 1 / 2	✅ 代码里写死（`fork_basket_cycle.py` 顶部常量，示教器读数，见第 7 章）	**不用教**	

完整搬运链：`抬升 L → （示教的 W）→ （示教的 T）→ 修剪位 1（写死）→ 修剪位 2（写死）`。没教 W/T 时运行端会直接拒绝：`teach file has no fixed fork_transfer_pose` / `has no loaded carry_waypoint`。 **下排专用**的 W/T 未教时不会拒绝——先沿用上排那对并打印 `WARNING: lower-row … not taught`（教一次下排盆的 W/T 即消除）。

键位速查（叉子示教）

键	作用	新旧
数字 / c / B	编号 / 清空 / 选中	同
F / I / L	叉接近 / 水平插入 / 竖直抬升（带校验）	同
W / T	全局载运点 / 暂放位	同
O	去固定观测位并留档	同
v	**切 A/B 侧（去该侧起始位 + 记住当前侧）**	新增
d	**记录当前位姿为当前侧下排观测位**	变化（旧版是"去固定参数位姿"）
g	**回当前侧下排观测位（验证用）**	新增
R	回检索位 + 释放 ServoJ（等 `Returned: max joint error=…` 打印）	同
Q	退出	同

三个"位"别搞混（`O` 去种球默认位是正常设计）：

键	去的位置	用途
`O`	待机位 = 种球默认位 `[-23.246, …]`	启动/收工回位并刷新记录；**不是看花篮的位置**
`v`	A / B 侧起始位（`-83.668… / 96.332…`，J1±180°）	教花篮的起点站位（即旧的"观测位"）；当前侧 A 时第一下 `v` 先去 B 侧，再按一下回 A
`g`	当前侧"下排观测位"（`d` 记录）	看下排盆（A1/A4/B1/B4）的码

进度检查：终端每次记录会打印 `Saved …`（下排盆会注明 `LOWER-row`）；核对清单用 `bash ~/Desktop/TL_lowerobs_test/scripts/list_taught.sh`。

> 提示：当流程处于 L / W / T 阶段（"花篮在叉上"）时，`d / g / v / O` 会被拒绝并提示 `the basket is on the fork; press R to return it before …`——先按 R 结束这盆的示教再操作。

11. 运行端验证（闭环）

11.1 种子（放种）

```
bash ~/Desktop/TL_lowerobs_test/scripts/run_seed_mqtt.sh
# 另一个终端发一条"下排盆"命令（任选一个已教的下排盆）：
mosquitto_pub -h 116.205.245.172 -u shapandevice -P 'qwer1234!!' -t node/arm/01/down \
  -m '{"cmd":"seed","targetId":"A1-05"}' -q 1
```

预期日志（下排盆）：`Basket number=N is on a lower row (A1/A4/B1/B4); moving to the side A lower observation pose to detect the code.` → 识别 → 放置完成（retCode 0）。

再发一条上排盆（如 `B2-06`）做回归：日志是 `Moving to the taught search pose for basket number=N.`，行为与旧版一致。

11.2 叉子（叉取）

```
bash ~/Desktop/TL_lowerobs_test/scripts/run_fork_mqtt.sh
# 另一个终端：
mosquitto_pub -h 116.205.245.172 -u shapandevice -P 'qwer1234!!' -t node/arm/01/down \
  -m '{"cmd":"trim_pick","targetId":"A3-08"}' -q 1     # 叉走
mosquitto_pub -h 116.205.245.172 -u shapandevice -P 'qwer1234!!' -t node/arm/01/down \
  -m '{"cmd":"trim_place","targetId":"A3-08"}' -q 1    # 放回
```

下排盆（A1/A4/B1/B4）会先去观测位：日志多出 `localizing marker at the lower-row observation pose...`。

trim_pick 完整日志顺序（上排盆为例）：

```
trim_pick: grabbing basket number=…
（识别）→ approach / insert / lift
carry: joint drive to … （W）
carry: joint drive to … （T）
carry: joint drive to trim position 1 joints=[…]
carry: T->P1
carry: P1->P2 high translate
carry: P2 lower: vertical
Basket number N is held at trim position 2. The fork has not released it; press R to
return it.
```

最后这句是给交互模式写的文案；**MQTT 模式此时应当发 `trim_place`**。

trim_place 返回： `hold2 → hold1 → T → W → L → I → F →` 检索位（日志以 `return: loaded …` 开头），结尾：

```
Basket returned to its original support; fork retracted; search pose reached.
Cycle complete at the observation (standby) pose.
```

状态文件时机：trim_pick 成功到位置 2 后 `~/tl_fork_teach/active_transfer.yaml` **存在**；trim_place 完整走完后**消失**。（中途退出会残留——见第 8 章。）

11.3 推荐的首次验证顺序

① 种子示教：记录 A/B 两侧下排观测位（d + g 验证）
② 种子运行：发 1 条下排盆 + 1 条上排盆，确认"下排走观测位 / 上排不变"
③ 叉子示教：O + 两侧观测位 + 教 1 个样例盆（B→F→I→L→W→T→R）
④ 叉子运行：trim_pick + trim_place 走一整圈（重点看新的修剪位 1 / 2）
⑤ 全部 OK 后再批量教盆；最后决定是否合并回 ~/Desktop/TL

12. 常见提示 / 报错对照

提示	含义与处理	
`Permission denied`	脚本没执行位：`chmod +x …/scripts/*.sh` 或改用 `bash 脚本` 方式	
`not visible or does not have enough stable samples`	按 B 太快/码看不清：停稳 1~2 秒重按（示教）	
`/tcp_pose is stale`（按 d 时）	臂没停稳：等 1~2 秒再按 d	
`side A/B lower observation pose is not recorded yet`	还没记录观测位：先拖到位按 d（示教端）	
`lower observation pose for side A is not recorded`（运行端 retCode 3）	**A 侧**没记录过观测位：到示教程序按 v→d 记录（**B 侧自 2026-10-07 起自动按 A 侧 J1+180° 推导，不再报错**；手动记录优先）	
`marker number N was not stable at the detection pose; got X samples`（运行端 retCode 3）	叉取时在识别位等约 10 秒没稳定看到码（被挡 / 光照 / 点位偏移）	失败后程序**自动回待机位**（2026-10-07 起，对齐种子流程）；排查后重新发同一条 `trim_pick` 重试
`the basket is on the fork; press R to return it before d/g/v/O`	叉上带盆阶段禁止这些操作：先按 R 放回	
`F->I horizontal travel is only …` / `I->L upward travel is only …` / `changed attitude by …`	拖得不够直/不够远：重新拖到位再按（阈值 20/30/60/25°）	
`teach file has no fixed fork_transfer_pose` / `no loaded carry_waypoint`	W/T 还没教：教第一盆时把 W、T 一起按了（下排盆会存成下排专用；下排未教前先用上排并打印 WARNING）	
`requested basket number X is not visible…`	输错号或该码当前不可见：核对编号/位姿	
`Active fork transfer state found …` / `Startup: a basket is still held` / trim_pick `retCode 2: a basket is already held; send trim_place first`	有未完成搬运的记账（原程序安全设计）。先看叉上有没有花篮：空的就删 `~/tl_fork_teach/active_transfer.yaml`；有的就发 `trim_place`（见第 8 章）	
`Command rejected/stopped: failed to check service availability: rcl node's context is invalid`	在动作**没走完**时按了 Ctrl+C：程序被关闭、动作停在半路。重启程序重做即可；动作期间不要按 Ctrl+C	
`MQTT disconnected: reason=None`	按 Ctrl+C 退出时的正常断开提示，不是错误	

`Returning with N direct ServoJ points.`	正常：一次流式关节运动开始（时长 ≈ N÷100 秒）；**走完的标志**是 `Returned: max joint error=… deg`	
`WARNING: … falling back to the joint interpolation path`	位置 1→位置 2 直线段不可达时自动改走关节插值（保护机制），偶发正常；频繁出现再反馈	
SciPy / NumPy `UserWarning`	环境版本提示，无害，忽略	
示教窗口运动后短暂无响应按键	可能是 `/tcp_pose` 短暂暂停：等几秒；持续卡死请记录日志反馈	

13. 数据文件与回退

文件	内容	说明
`~/tl_mock_pick_place/basket_aruco.yaml`	种子示教数据（新增 `lower_observation_pose_a/_b`）	新旧程序共用；新键只**追加**，不破坏旧数据
`~/tl_fork_teach/fork_basket_teach.yaml`	叉子示教数据（同样新增 `lower_observation_pose_a/_b`）	每次写入自动备份到 `archive/`
`~/tl_fork_teach/active_transfer.yaml`	**作业状态文件**（进行中的搬运记账）	trim_place 完成自动删除；残留时的处理见第 8 章
`fork_basket_cycle.py` 顶部常量	修剪位 1 / 2 固定点位	修改方法见第 7.5 节（旧值在第 7.4 节备查）

- 旧程序**不读**这两个"下排观测位"新键，所以不跑新流程时旧程序照用不受影响（但别和新程序同时跑）。
- **回退**：不再执行 `TL_lowerobs_test` 里的脚本即可；或直接删除整个 `TL_lowerobs_test` 文件夹。修剪点位如需回退用第 7.4 节旧值 + 第 7.5 节脚本。
- 验证通过后如需**合并回 `~/Desktop/TL`**（并提交 git），另行处理。

14. 操作纪律与快捷索引

纪律（每一条都是踩过的坑）：

- E-stop 放在手边；同一时间只开一个会发运动的程序。
- **动作没走完别按 Ctrl+C**；要停等空闲，急停用物理 E-stop。
- 按 `d` 前先把臂拖稳停好；按完核对 `side=A/B`。
- 换点位 / 换版本后，先空载低速试一圈再上真花篮。

快捷索引：

```
测新版跑起来了没：启动打印有 "Current side:" 就是新版
记录观测位：       拖到位 → 停稳 → d（核对 side=A/B）→ g 验证
切侧：             v
查修剪点位：       grep -n
"TRIM_HOLD" …/src/tl_fork_teach/tl_fork_teach/fork_basket_cycle.py
清搬运残留：       cat 后 rm ~/tl_fork_teach/active_transfer.yaml（先确认叉上无花篮）
动作完成标志：     "Returned: max joint error=…" / "Cycle complete …"
发下排盆命令：     A1-xx / A4-xx / B1-xx / B4-xx
发上排盆命令：     A2-xx / A3-xx / B2-xx / B3-xx（行为同旧版；A3 排有"贴反修正"，见第 15 章）
```

15. 修正：A3 排码贴反（`--reversed-rows A3`）

背景：A3 排的 ArUco 码贴反了（本该 ID48→ID71，实际贴成 ID71→ID48），导致"输入的位置号"和"格子上实际的码"对不上——例如发 `A3-08`，程序实际去找 ID55 对应的已教点（它在排后面），所以机械臂跑到后面去了。

改法：4 个脚本（种子/叉子 的示教+运行）默认带 `--reversed-rows A3`：**输入的位置号会自动镜像成该格实际贴的码**。对应关系：

输入	实际使用	输入	实际使用
A3-01	ID71	A3-17	ID55
A3-08	ID64	A3-18	ID54
A3-11	ID61	A3-24	ID48
A3-14	ID58		

- 只影响 A3 一排；其它排原样不变。镜像是对称的（套两次回原值），**不要对已转换过的编号重复套用**。
- 覆盖 / 其它排 / 关闭：`TL_REVERSED_ROWS=B2 bash scripts/run_seed_mqtt.sh`；多个排用空格 `TL_REVERSED_ROWS="A3 B2"`；关闭用 `TL_REVERSED_ROWS=none`（置空会回退到默认 A3）。
- **已教 A3 盆对应命令变化**：55 → **A3-18**、56 → **A3-17**、62 → **A3-11**（旧写法 A3-07 / A3-08 / A3-14 修正后会指向没教过的格，被拒绝）；要补教 A3-08 / A3-14，用新版示教程序按老流程教（输入 56 / 62 会正确匹配第 8 / 14 格上的码 ID64 / ID58）。
- **查点工具已对齐**：`list_taught.sh` 的 A3 位置列直接显示可用命令名（编号 55 → A3-18；`--basket A3-17` 按换算查询）。
- **日志对照**：程序启动打印 `Reversed-row code mapping active: ['A3'] …`；每条 A3 命令打印 `Reversed-row mapping: targetId=… -> actual code number …`；示教端输入 A3 编号时打印同样一行。看到这些即为修正生效。
- **范围**：本修正只在验证包 `TL_lowerobs_test` 的脚本里；老版 `~/Desktop/TL` 没有（A3 的教与跑请用新版脚本）。